

HospiTrack Architecture Analysis

Analysis Date: December 15, 2024

Repository: <https://github.com/JZembower/HospiTrack>

Project Goal: Enhance HospiTrack to create a publicly accessible hospital finder with advanced sorting, triage demo, and deployment-ready configuration.

Table of Contents

1. [Executive Summary](#)
 2. [Current Codebase Structure](#)
 3. [Existing API Endpoints](#)
 4. [Frontend Structure](#)
 5. [Dataset Analysis](#)
 6. [Docker & Deployment Configuration](#)
 7. [Identified Gaps](#)
 8. [Recommendations for New Features](#)
 9. [Implementation Roadmap](#)
-

Executive Summary

Current State

HospiTrack is a functional FastAPI-based hospital finder application with the following capabilities:

- **Backend:** FastAPI with 4 main endpoints serving hospital data
- **Frontend:** Single-page vanilla HTML/CSS/JS interface
- **Data:** 4,088 US hospitals with geolocation, quality metrics, and complaint-adjusted scoring
- **Deployment:** Docker-ready with docker-compose configuration
- **Features:** Distance filtering, multi-criteria sorting (quality, wait time, rating, mortality), state filtering

Enhancement Goals

1. **Multi-page navigation:** Home → Results → Explore → Company Demo
 2. **Triage system:** Rule-based + optional ML model demo
 3. **Enhanced UX:** Responsive design, ranking explanations, URL state management
 4. **Production readiness:** Render deployment, geocoding optimization, medical disclaimers
-

Current Codebase Structure

Directory Layout

```
HospiTrack/
├── main.py                # FastAPI application & routes
├── modules/
│   ├── data_loader.py    # Dataset loading & caching
│   ├── sorting_logic.py  # Mortality parsing & complaint adjustment
│   ├── geolocation.py    # Geocoding & distance calculation
│   └── map_display.py     # Folium map rendering
├── static/
│   └── index.html        # Current single-page UI
├── data/
│   ├── us_er.parquet     # Cached hospital dataset (268KB)
│   └── us_er_transformed.csv # Source CSV (3.3MB)
├── Dockerfile            # Python 3.11-slim production image
├── docker-compose.yml    # Local development stack
├── requirements_fastapi.txt # Python dependencies
├── Additional directories:
│   ├── Data Transformation/ # ETL/scraping scripts
│   ├── tools/               # Utility scripts
│   └── .venv/               # Virtual environment
```

Module Analysis

main.py (319 lines)

Key Components:

- FastAPI app with CORS middleware (permissive for dev)
- Background thread data loading for fast startup
- Routes: / (root), /healthz, /map, /api/hospitals, /api/states
- Sort options: adjusted_quality_points, detail_avg_time_in_ed_minutes, detail_overall_patient_rating, mortality
- Location resolution: Accepts address string or lat/lon coordinates
- Default fallback: Chicago (41.8781, -87.6298)

Current Limitations:

- No multi-page routing structure
- No triage functionality
- No user preference/history management
- Limited error handling for geocoding failures
- No rate limiting for geocoding API

modules/data_loader.py (149 lines)

Key Functions:

- load_data(path) : Smart loader with Parquet caching
- build_parquet_cache() : Converts CSV → Parquet with column selection
- _ensure_lat_lon() : Batch geocoding via pgeocode (offline ZIP lookup)
- _clean_repeated_headers() : Data quality enforcement

Strengths:

- Efficient caching strategy (268KB Parquet vs 3.3MB CSV)

- Vectorized geocoding for performance
- Automatic data cleaning

Limitations:

- No support for incremental data updates
- No validation of data freshness
- API_COLUMNS hardcoded (limits extensibility)

modules/sorting_logic.py (94 lines)

Key Functions:

- `parse_mortality(val)` : Parses “46% better” → (type, numeric_value)
- `prepare_mortality_sort()` : Adds sorting columns (mortality_order, mortality_sort_value)
- `apply_complaint_adjustment()` : Maps user complaints to specialized quality columns

Complaint Mapping:

```
{
  "Overall": "total_quality_points",
  "Heart Attack/Chest Pain": "adj_total_heartattack",
  "Stroke/Slurred Speech/Facial Droop": "adj_total_stroke",
  "Shortness of Breath/Cough/Fever": "adj_total_pneu"
}
```

Strengths:

- Smart mortality parsing with ordinal ranking
- Complaint-specific quality scoring

Gaps:

- Limited to 3 complaint categories
- No triage severity scoring

modules/geolocation.py (102 lines)

Key Functions:

- `safe_geocode()` : Geopy/Nominatim with retry logic (3 attempts, 2s delay)
- `validate_location()` : Optional Midwest-only filtering (disabled for nationwide)
- `add_distance()` : Haversine distance calculation via geopy.geodesic

Current Geocoding:

- Service: OpenStreetMap Nominatim (free)
- Rate limit: Manual 2s delays + timeout
- User-Agent: “hospi_track_app”

Issues:

- No caching of geocoded addresses (repeated queries waste API calls)
- Nominatim usage policy concerns (1 req/sec limit)
- No fallback geocoding service

modules/map_display.py (57 lines)

Key Functions:

- `render_map_html()` : Creates Folium interactive map with marker clustering

Features:

- Leaflet-based with CartoDB Positron tiles

- User location blue marker + hospital red markers
- Popup: Name, address, distance, quality, ED time, rating, mortality

Limitations:

- Max 5,000 points (hard-coded)
- No custom icon differentiation by priority/quality
- No real-time filtering on map

Existing API Endpoints

1. GET / (Root)

Purpose: Landing page with links

Response: HTML with status/docs/UI links + examples

2. GET /healthz

Purpose: Health check for monitoring/deployment

Response:

```
{"status": "ready" || "starting" || "error"}
```

Status Codes:

- 200: Ready
- 503: Data loading
- 500: Startup error

3. GET /map

Purpose: Interactive Folium map view

Query Parameters:

Parameter	Type	Default	Description
address	string	""	Free-form address or "City, ST"
lat	float	None	User latitude
lon	float	None	User longitude
sort	string	adjusted_quality_points	Sort criteria
complaint	string	Overall	Complaint type
top_k	int	50	Max results (1-1000)
within_km	float	300.0	Radius filter (1-10000)
state	string	None	Two-letter state code

Response: HTML page with embedded Leaflet map

Logic:

1. Resolve user location (address → geocode OR lat/lon)
2. Apply complaint adjustment to quality scores
3. Filter by distance + optional state
4. Sort by selected criteria
5. Take top K results
6. Render Folium map with markers

4. GET /api/hospitals

Purpose: JSON API for hospital search (primary data endpoint)

Query Parameters: Same as /map (except different defaults)

Parameter	Type	Default	Description
top_k	int	50	Max results (1-2000)
within_km	float	200.0	Radius filter

Response:

```
{
  "count": 25,
  "results": [
    {
      "hospital_name": "...",
      "detail_address": "...",
      "detail_city": "...",
      "detail_state": "IL",
      "detail_zip": 60611,
      "lat": 41.8955,
      "lon": -87.6217,
      "distance_km": 1.23,
      "total_quality_points": 17,
      "adjusted_quality_points": 15,
      "detail_avg_time_in_ed_minutes": 145,
      "detail_overall_patient_rating": "GOOD",
      "detail_mortality_overall_text": "4% better",
      "Top_Procedures": "..."
    }
  ]
}
```

5. GET /api/states

Purpose: List of available state codes for dropdown population

Response:

```
{
  "states": ["AK", "AL", "AR", "AZ", "..."]
}
```

Logic: Extracts unique `detail_state` values from dataset, fallback to hardcoded list

Frontend Structure

Current UI: `/static/index.html` (193 lines)

Layout:

Features:

- Grid-based responsive controls (2fr-1fr-1fr-1fr-1fr)
- Async JavaScript fetch API
- URLSearchParams for query building
- Table with “View Map” links per row
- Auto-load states dropdown on page load
- Default search on page load (falls back to Chicago)

Styling:

- System fonts (system-ui, -apple-system, Roboto)
- Background: #fbf9f7 (off-white)
- Mobile responsive: 2-column grid at <960px
- Sticky table header

JavaScript Functions:

- `loadStates()` : Fetches `/api/states` and populates dropdown
- `search()` : Main search function calling `/api/hospitals`
- `qsParams()` : `URLSearchParams` builder
- `toFixedOrDash()` : Numeric formatter with fallback

Current Limitations:

- No multi-page navigation (SPA architecture needed)
- No URL state management (no browser history)
- No complaint selector in UI (hard-coded to “Overall”)
- No triage intake form
- No ranking explanation tooltips
- No loading spinners (just text)
- No medical disclaimers
- No “Explore” mode for US-wide browsing

Dataset Analysis

1. Hospital Dataset (data/us_er.parquet)

Size: 4,088 hospitals | 14 columns | 268KB

Coverage: All 50 states + DC (51 unique states)

Schema

Column	Type	Non-Null	Null %	Description
hospital_name	object	4,088 (100%)	0.0%	Hospital/clinic name
detail_address	object	4,062 (99.4%)	0.6%	Street address
detail_city	category	4,088 (100%)	0.0%	City name
detail_state	category	4,088 (100%)	0.0%	Two-letter state code
detail_zip	float64	4,062 (99.4%)	0.6%	ZIP code
lat	float64	3,554 (86.9%)	13.1%	Latitude (WG-S84)
lon	float64	3,554 (86.9%)	13.1%	Longitude (WG-S84)
total_quality_points	int64	4,088 (100%)	0.0%	Overall quality score (6-22)
detail_avg_time_in_ed_minutes	float64	3,696 (90.4%)	9.6%	ED wait time (45-587 min)
detail_overall_patient_rating	object	4,062 (99.4%)	0.6%	Rating (VERY GOOD, GOOD, AVERAGE, POOR)
detail_mortality_overall_text	object	3,173 (77.6%)	22.4%	Mortality comparison (e.g., "4% better")
adj_total_heartattack	object	4,088 (100%)	0.0%	Heart attack quality points
adj_total_stroke	object	4,088 (100%)	0.0%	Stroke quality points
adj_total_pneu	object	4,088 (100%)	0.0%	Pneumonia quality points

Key Statistics

Geographic Distribution (Top 5):

- Texas: 340 hospitals (8.3%)

- California: 271 (6.6%)
- Florida: 169 (4.1%)
- Pennsylvania: 151 (3.7%)
- New York: 147 (3.6%)

ED Wait Time:

- Mean: 161.8 minutes (~2.7 hours)
- Median: 154 minutes
- Range: 45-587 minutes
- 25th %ile: 122 min | 75th %ile: 195 min

Patient Rating Distribution:

- AVERAGE: 2,002 (49.3%)
- GOOD: ~30%
- VERY GOOD: ~15%
- POOR: ~5%

Quality Points:

- Mean: 16.0
- Median: 16
- Range: 6-22
- SD: 2.07

Mortality Text (Top 5):

- 2% worse: 138
- 4% worse: 126
- 7% worse: 125
- 1% worse: 124
- 3% worse: 119

Geolocation Quality:

- Both lat/lon valid: 3,554 (86.9%)
- Missing coords: 534 (13.1%)
- Note: Missing coords likely due to invalid/incomplete ZIP codes

Data Quality Issues

1. **13% missing geolocation** → Affects mapping and distance filtering
2. **22% missing mortality data** → Limits mortality-based sorting
3. **Complaint-adjusted columns are text** → Need parsing/conversion for ML use
4. **Patient rating is categorical** → Need encoding for numeric sorting

2. Triage Dataset (/home/ubuntu/Uploads/data.csv)

Size: 1,267 patients | 24 columns | 126KB

Source: Korean Triage and Acuity Scale (KTAS) dataset

Format: Semicolon-delimited CSV (latin-1 encoding)

Schema Overview

Category	Columns	Key Details
Demographics	Group, Sex, Age	Age: 18-92, Sex: 1=Male/2=Female
Arrival Context	Patients per hour, Arrival mode	ED crowding + transport method
Chief Complaint	Chief_complain, Injury	417 unique complaints
Vital Signs	SBP, DBP, HR, RR, BT, Saturation	Mostly complete except O2 sat (54% missing)
Pain Assessment	Mental, Pain, NRS_pain	Numeric Rating Scale 0-10
Triage Outcomes	KTAS_RN, KTAS_expert	RN vs expert triage levels (1-5)
Clinical	Diagnosis in ED, Disposition	583 unique diagnoses
Quality Metrics	Error_group, mistriage, Length of stay	Triage accuracy metrics

Target Variables for ML

1. KTAS_RN (Registered Nurse Triage):

Level	Count	%	Severity
4 (Less Urgent)	501	39.5%	Low risk
3 (Urgent)	447	35.3%	Moderate risk
2 (Emergent)	214	16.9%	High risk
5 (Non-Urgent)	87	6.9%	Minimal risk
1 (Immediate)	18	1.4%	Life-threatening

2. KTAS_expert (Expert Triage - Ground Truth):

Level	Count	%
3	487	38.4%
4	459	36.2%
2	220	17.4%
5	75	5.9%
1	26	2.1%

3. mistriage (Triage Error Indicator):

- 0 (Correct): 1,081 (85.3%)
 - 2 (Undertriage): 131 (10.3%)
 - 1 (Overtriage): 55 (4.3%)

Top Chief Complaints

Complaint	Count	Typical KTAS
abd pain	72	3-4
dyspnea	60	2-3
dizziness	59	3-4
fever	45	3-4
ant. chest pain	44	2-3
Open Wound	31	2-4
headache	30	3-4

Vital Sign Completeness

Vital	Non-Null	Null %
SBP	1,267	0%
DBP	1,267	0%
HR	1,267	0%
RR	1,267	0%
BT	1,267	0%
Saturation	579	54.3%

Note: Vital signs stored as text (need parsing for ML)

ML Model Potential

Strengths:

1. **High-quality labeled data** (expert triage as ground truth)
2. **Rich feature set** (24 features including vitals, symptoms, context)
3. **Real-world clinical data** (actual ED triage outcomes)
4. **Low missing data** (2.27% overall, 100% complete for 45.5% of rows)
5. **Balanced target distribution** (KTAS 2-4 are well-represented)

Challenges:

1. **Small dataset** (1,267 samples - risk of overfitting)
2. **Text parsing needed** (chief complaints, vitals stored as strings)
3. **Korean medical terms** (some chief complaints in Korean, need translation/mapping)
4. **Class imbalance** (KTAS 1 = only 18 cases)
5. **Cultural specificity** (KTAS may not translate directly to US ESI system)

Recommended Approach:

- **Rule-based baseline** (default for production): Use vitals + chief complaint keywords
- **ML model demo** (optional feature): Train multi-class classifier (Random Forest/XGBoost)
- **Clearly label** as “Demo Mode - For Educational Purposes Only”
- **Synthetic data augmentation** (SMOTE for minority classes)
- **Feature engineering**: NLP on chief complaints, age binning, vital sign severity scores

Docker & Deployment Configuration

Dockerfile Analysis

Base Image: `python:3.11-slim`**Dependencies:**

- build-essential, gcc, libpq-dev (for compiled extensions)
- debugpy (VS Code debugging)
- Gunicorn + Uvicorn workers (production ASGI server)

Configuration:

```
ENV PYTHONUNBUFFERED=1
ENV HOSPITRACK_DATA_PATH=/app/data
ENV HOST=0.0.0.0
ENV PORT=8000
EXPOSE 8000 5678 # App + debugger
```

CMD:

```
gunicorn -k uvicorn.workers.UvicornWorker main:app \
  -w 2 -b 0.0.0.0:8000 --log-level info
```

Image Size Optimization:

- Slim base image (~150MB vs ~1GB for full Python image)
- Requirements layer caching
- Single-stage build (could optimize further with multi-stage)

docker-compose.yml

Services:

- `hospitrack` : Main app container

Volumes:

- `./data:/app/data:ro` (read-only mount for dataset)
- `./static:/app/static` (live reload for frontend dev)

Ports:

- 8000:8000 (HTTP)
- 5678:5678 (Debugger)

Restart Policy: `unless-stopped`

Production Deployment Gaps

Current Issues:

1. **CORS:** Permissive `allow_origins=["*"]` - needs restriction for production
2. **No rate limiting** on geocoding endpoints
3. **No caching headers** for static assets
4. **No HTTPS configuration** (expects reverse proxy)
5. **No health check intervals** in docker-compose (Render requires specific format)
6. **No environment-specific configs** (dev vs staging vs prod)
7. **Data path hardcoded** (should use environment variable)
8. **No log rotation** or structured logging
9. **No monitoring/observability** (no Prometheus metrics, APM)

Render-Specific Requirements:

- **Health check endpoint:** ☒ Already have `/healthz`
 - **PORT environment variable:** ☒ Supported via `$PORT`
 - **Persistent storage:** ☒ Need to bundle data in image or use external storage
 - **Build command:** `docker build`
 - **Start command:** CMD in Dockerfile (already configured)
 - **render.yaml:** ☒ Missing (need for Infrastructure as Code)
-

Identified Gaps

1. Frontend Architecture Gaps

Gap	Current State	Required State
Multi-page navigation	Single HTML page	Home, Results, Explore, Demo pages
URL state management	No query params in URL	Search state persisted in URL for sharing
Complaint selector	Hardcoded to “Overall”	Dropdown with all complaint types
Triage intake form	Does not exist	Multi-step form with symptom input
Ranking explanations	No tooltips/modals	Info icons with detailed scoring logic
Loading states	Text only	Proper spinners/skeletons
Medical disclaimers	None	Prominent disclaimers on all pages
Responsive design	Basic (2-col grid)	Mobile-first with proper breakpoints
Accessibility	No ARIA labels	WCAG 2.1 AA compliance

2. Backend API Gaps

Gap	Current State	Required State
Triage endpoint	Does not exist	/api/triage with rule-based + ML options
Geocoding cache	No caching	Redis/in-memory cache for addresses
Rate limiting	None	Per-IP rate limits (especially for geocoding)
Input validation	Basic regex	Pydantic models for all inputs
Error responses	Generic	Structured error responses with codes
Search history	None	Optional user preference storage
Analytics	None	Usage tracking (page views, searches)

3. Data & ML Gaps

Gap	Current State	Required State
Triage model	None	Rule-based + optional ML classifier
Model versioning	N/A	Model registry with A/B testing
Feature store	Raw columns only	Engineered features (severity scores, etc.)
Data validation	Basic checks	Great Expectations or Pydantic schemas
Dataset updates	Manual Parquet rebuild	Automated pipeline with versioning
Synthetic data	None	Generated triage scenarios for ML

4. Deployment & Operations Gaps

Gap	Current State	Required State
Render config	None	<code>render.yaml</code> with health checks
Environment configs	Single Dockerfile	Dev/staging/prod env files
CORS policy	Permissive	Restricted to production domain
Logging	Basic <code>print()</code>	Structured JSON logs (Python logging)
Monitoring	None	Uptime monitoring + error tracking
CI/CD	Manual	GitHub Actions for testing + deployment
Secrets management	Hardcoded	Environment variables + Render secrets

5. Documentation Gaps

Gap	Current State	Required State
Deployment guide	Docker-only	Step-by-step Render deployment
API docs	FastAPI /docs (auto)	Enhanced with examples + rate limits
User guide	None	How to use triage, interpret rankings
Data update SOP	README mentions	Detailed procedure with scripts
Medical disclaimers	None	Legal notice + limitations document

Recommendations for New Features

Priority 1: Core User Experience (Week 1-2)

1.1 Multi-Page Navigation

Implementation:

- Create `/static/home.html`, `/static/results.html`, `/static/explore.html`, `/static/demo.html`
- Add navigation header component (included in all pages)
- Use vanilla JS router or simple href links with shared state (localStorage)

File Structure:

```
static/
├── index.html          # → Redirect to home.html
├── home.html           # Landing page with search
├── results.html        # Search results + map
├── explore.html        # US-wide browsing
├── demo.html           # Triage demo intake
├── css/
│   └── styles.css      # Shared styles
├── js/
│   ├── api.js          # API client wrapper
│   ├── state.js        # URL state management
│   └── utils.js         # Shared utilities
├── components/
│   ├── nav.html        # Header template (or JS component)
│   └── footer.html      # Footer template
```

Home Page (home.html):

- Hero section: "Find the Right ER for Your Emergency"
- Symptom/priority input (dropdown or autocomplete)
- Location input (address or "Use My Location" button)
- CTA: "Find Hospitals" → results.html with query params

Results Page (results.html):

- Search filters (same as current index.html)
- Map view (Folium iframe or Leaflet.js integration)
- Table view (sortable, paginated)
- "Back to Search" / "Refine Search" options

Explore Page (explore.html):

- State dropdown + city filter
- No location requirement (show all in state)
- Browse by specialty/complaint type
- Educational content: "What to expect in an ER"

Demo Page (demo.html):

- Triage intake form (multi-step)
- Recommended priority level (KTAS/ESI mapping)
- Suggested hospitals sorted by priority + quality
- Clear disclaimer: "Educational Demo Only"

1.2 URL State Management

Implementation:

- Use `URLSearchParams` to encode search state
- On page load: parse query string → populate form → trigger search
- On search: update URL with `history.pushState()`

Example URL:

```
/results.html?address=Chicago%2C+IL&complaint=chest_pain&sort=quality&within_km=50
```

Benefits:

- Shareable links
- Browser back/forward navigation
- SEO-friendly (if later adding server-side rendering)

1.3 Complaint Selector in UI

Current: Hardcoded to "Overall"

New: Dropdown with options:

- Overall
- Chest Pain / Heart Attack
- Stroke / Slurred Speech / Facial Droop
- Shortness of Breath / Cough / Fever

Location: Add to home.html search form + results.html filters

1.4 Medical Disclaimers

Placement:

- Footer on all pages: "Not for medical emergencies. Call 911 for life-threatening conditions."
- Modal on first visit: "This tool is informational only. Always consult medical professionals."
- Triage demo: Banner at top: "EDUCATIONAL DEMO - NOT FOR ACTUAL TRIAGE USE"

Content:

MEDICAL DISCLAIMER

This **tool** provides informational estimates based on publicly available data. It **is** NOT a substitute **for** professional medical advice, diagnosis, **or** treatment. For medical emergencies, call 911 **or** go to the nearest emergency room. Data may **not** reflect real-time conditions. Hospital quality **and** wait times vary.

Priority 2: Triage System (Week 2-3)

2.1 Rule-Based Triage (Default, Production-Ready)

Algorithm Design:

Inputs:

1. Chief complaint (text or dropdown)
2. Pain level (0-10 scale)
3. Vital signs (optional: BP, HR, RR, temp, O2 sat)
4. Age (optional)
5. Symptoms checkboxes (bleeding, altered mental status, shortness of breath, etc.)

Output: KTAS level (1-5) mapped to US ESI equivalent

Rule Logic (simplified):

```
def rule_based_triage(symptoms, vitals, pain_level, age):
    # Level 1: Immediate (life-threatening)
    if any([
        "cardiac arrest" in symptoms,
        "not breathing" in symptoms,
        vitals.get('hr') < 40 or vitals.get('hr') > 150,
        vitals.get('sbp') < 70,
        vitals.get('o2sat') < 85
    ]):
        return 1, "Immediate - Life-threatening"

    # Level 2: Emergent (high risk)
    if any([
        "chest pain" in symptoms and age > 40,
        "severe bleeding" in symptoms,
        pain_level >= 9,
        vitals.get('sbp') < 90,
        vitals.get('rr') > 30 or vitals.get('rr') < 10
    ]):
        return 2, "Emergent - High risk"

    # Level 3: Urgent (moderate risk)
    if any([
        pain_level >= 7,
        "fever" in symptoms and vitals.get('temp') > 39.5,
        "vomiting" in symptoms,
        age > 75
    ]):
        return 3, "Urgent - Moderate risk"

    # Level 4: Less urgent (low risk)
    if pain_level >= 4:
        return 4, "Less Urgent - Low risk"

    # Level 5: Non-urgent (minimal risk)
    return 5, "Non-Urgent - Minimal risk"
```

Implementation:

- New module: `modules/triage_rules.py`
- New endpoint: `POST /api/triage` (accepts JSON with symptoms/vitals)
- Response includes: level, description, recommended hospitals

2.2 ML Triage Model (Optional Demo)

Model Type: Multi-class classifier (Random Forest or XGBoost)

Training Pipeline:

```
# Preprocessing
- Parse vital signs from text to numeric
- Encode chief_complain using TF-IDF or pre-trained medical embeddings
- One-hot encode categorical features (sex, arrival mode)
- Handle missing O2 saturation (median imputation)

# Feature Engineering
- Age bins: <18, 18-40, 40-65, 65-75, >75
- Vital severity scores (z-scores vs normal ranges)
- Chief complaint categories (pain, respiratory, cardiac, trauma, etc.)

# Target Variable
- KTAS_expert (ground truth)
- Handle class imbalance with SMOTE or class weights

# Model Selection
- Baseline: Logistic Regression (interpretable)
- Advanced: Random Forest (handles non-linearity)
- Best: XGBoost with hyperparameter tuning

# Evaluation Metrics
- Accuracy (overall correctness)
- Macro F1-score (class balance)
- Confusion matrix (undertriage vs overtriage)
- Clinical metric: Critical undertriage rate (labeling Level 1 as 4-5)
```

Deployment:

- Serialize model: `models/triage_classifier.pkl` (pickle or joblib)
- Load at startup: `triage_model = joblib.load('models/triage_classifier.pkl')`
- Inference endpoint: `POST /api/triage?mode=ml`
- Clearly label in UI: “ ML Demo Mode (Experimental)”

Synthetic Data Generation:

- Use ChatGPT/Claude to generate realistic triage scenarios
- Create 500+ synthetic cases with labels
- Mix real KTAS data + synthetic data for training
- Validate on held-out real data only

2.3 Triage-Enhanced Hospital Recommendations

Logic:

1. User completes triage intake → gets priority level (1-5)
2. Backend filters hospitals by:
 - Distance (within user-specified km)
 - Complaint-adjusted quality (map KTAS → complaint type)
 - ED wait time (prioritize lower for urgent cases)
3. Sort by weighted score:

python

```
score = 0.5 * (quality / max_quality) +
        0.3 * (1 - wait_time / max_wait_time) +
        0.2 * (1 - distance / max_distance)
```

4. Display top 10 with reasoning:

-  “Best overall quality for chest pain cases”
-  “Shortest average wait time (45 min)”
-  “Closest to your location (1.2 km)”

Priority 3: Production Readiness (Week 3-4)

3.1 Geocoding Optimization

Problem: Nominatim free tier = 1 request/second, no caching, unreliable for production

Solution 1: Caching (Immediate)

```
# In-memory cache with TTL
from functools import lru_cache
from cachetools import TTLCache

geocode_cache = TTLCache(maxsize=1000, ttl=86400) # 1000 entries, 24hr TTL

@lru_cache(maxsize=500)
def cached_geocode(address: str):
    if address in geocode_cache:
        return geocode_cache[address]
    result = safe_geocode(address)
    geocode_cache[address] = result
    return result
```

Solution 2: Rate Limiting

```
from slowapi import Limiter
from slowapi.util import get_remote_address

limiter = Limiter(key_func=get_remote_address)
app.state.limiter = limiter

@app.get("/api/hospitals")
@limiter.limit("10/minute") # 10 searches per minute per IP
async def api_hospitals(...):
    pass
```

Solution 3: Fallback Services (Production)

- Use Nominatim as primary (free)
- Fallback to PositionStack API (free tier: 10k requests/month)
- Cache all results in Redis (persistent across restarts)

Solution 4: Pre-geocode Common Locations

- Generate geocode cache for top 1000 US cities
- Store in `data/city_geocode_cache.json`
- Load at startup → instant results for common queries

3.2 Render Deployment Guide

Create `render.yaml` :

```

services:
- type: web
  name: hospitracker
  env: docker
  plan: starter # $7/month
  region: oregon
  healthCheckPath: /healthz
  envVars:
    - key: PORT
      value: 8000
    - key: HOSPITRACK_DATA_PATH
      value: /app/data
    - key: PYTHON_VERSION
      value: 3.11
  dockerfilePath: ./Dockerfile
  dockerContext: .

```

Deployment Steps:

1. Create Render account + connect GitHub repo
2. Push `render.yaml` to repo
3. Render auto-detects and builds Docker image
4. Set custom domain (optional): `hospitracker.onrender.com` → `yourdomain.com`
5. Configure environment variables in Render dashboard
6. Enable auto-deploy on git push to main branch

Production Environment Variables:

- `ALLOWED_ORIGINS` : Comma-separated list of domains for CORS
- `LOG_LEVEL` : `INFO` (vs `DEBUG` in dev)
- `GEOCODE_CACHE_SIZE` : 5000 (increase cache size)

3.3 CORS Policy Update

Current:

```
allow_origins=["*"] # ❌ Permissive
```

Production:

```

import os
allowed = os.getenv("ALLOWED_ORIGINS", "http://localhost:8000").split(",")
app.add_middleware(
    CORSMiddleware,
    allow_origins=allowed, # ✅ Restricted
    allow_credentials=True,
    allow_methods=["GET", "POST"],
    allow_headers=["*"],
)

```

3.4 Structured Logging

Replace `print()` with logging:

```

import logging
import json

logging.basicConfig(
    level=os.getenv("LOG_LEVEL", "INFO"),
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
    handlers=[logging.StreamHandler()]
)
logger = logging.getLogger("hospitrack")

# Structured logging for production
def log_search(address, sort, top_k, results_count):
    logger.info(json.dumps({
        "event": "hospital_search",
        "address": address,
        "sort": sort,
        "top_k": top_k,
        "results": results_count,
        "timestamp": time.time()
    })))

```

Benefits:

- Parse logs in Render dashboard
- Export to external monitoring (Datadog, Sentry)
- Track usage patterns and errors

Priority 4: Enhanced UX (Week 4+)**4.1 Ranking Explanations****Implementation:**

- Add info icons (i) next to table headers
- Tooltip on hover with explanation
- Modal with detailed methodology

Example Tooltips:

- **Quality Points:** "Composite score based on Medicare Hospital Compare metrics including patient outcomes, safety, and care coordination."
- **ED Wait Time:** "Average time from arrival to seeing a physician, based on hospital-reported data. Actual times vary by severity and time of day."
- **Mortality:** "Comparison to national average for 30-day mortality rates. 'Better' means lower mortality than expected."

4.2 Loading States & Skeletons

Current: "Searching..." text

New:

- Spinner animation (CSS or SVG)
- Skeleton loaders for table rows (gray placeholder boxes)
- Progress bar for multi-step triage form

Example:

```
<div class="skeleton-loader">
  <div class="skeleton-row"></div>
  <div class="skeleton-row"></div>
  <div class="skeleton-row"></div>
</div>
```

```
.skeleton-row {
  height: 40px;
  background: linear-gradient(90deg, #f0f0f0 25%, #e0e0e0 50%, #f0f0f0 75%);
  background-size: 200% 100%;
  animation: loading 1.5s infinite;
  margin: 8px 0;
  border-radius: 4px;
}

@keyframes loading {
  0% { background-position: 200% 0; }
  100% { background-position: -200% 0; }
}
```

4.3 Accessibility Improvements

WCAG 2.1 AA Compliance:

- ☒ Color contrast ratio $\geq 4.5:1$ for text
- ☒ Keyboard navigation (tab order, focus indicators)
- ☒ ARIA labels for all interactive elements
- ☒ Alt text for images/icons
- ☒ Screen reader announcements for dynamic content

Example:

```
<button aria-label="Search for hospitals" id="searchBtn">Search</button>
<table role="table" aria-label="Hospital search results">
  <thead>
    <tr role="row">
      <th role="columnheader" scope="col">Hospital</th>
      ...
    </tr>
  </thead>
</table>
```

4.4 Mobile Optimization

Current: 2-column grid at <960px

Enhanced:

- Mobile-first CSS (start with mobile, add desktop media queries)
- Touch-friendly buttons (min 44x44px)
- Collapsible filters on mobile
- Bottom sheet for map controls
- Swipeable table rows for actions

Breakpoints:

```

/* Mobile: 320px - 767px */
/* Tablet: 768px - 1023px */
/* Desktop: 1024px+ */

@media (max-width: 767px) {
  .controls {
    grid-template-columns: 1fr; /* Single column */
  }
  table {
    font-size: 12px; /* Smaller text */
  }
}

```

Implementation Roadmap

Phase 1: Foundation (Week 1)

Goal: Multi-page architecture + URL state management

Tasks:

1. ☒ Clone repository and analyze codebase
2. Create file structure:
 - /static/home.html , /results.html , /explore.html , /demo.html
 - /static/css/styles.css
 - /static/js/api.js , /state.js , /utils.js
3. Implement shared navigation component
4. Add URL state management (search params)
5. Update /api/hospitals to accept complaint parameter
6. Add medical disclaimers to all pages
7. Test navigation flow: Home → Results → Explore → Demo

Deliverables:

- Working 4-page navigation
- Shared styles and API client
- URL-based state persistence

Phase 2: Triage System (Week 2-3)

Goal: Rule-based triage + optional ML demo

Tasks:

1. Design triage intake form (multi-step):
 - Step 1: Chief complaint dropdown + free text
 - Step 2: Pain level (0-10 slider)
 - Step 3: Vital signs (optional fields)
 - Step 4: Symptom checkboxes (bleeding, confusion, etc.)
2. Implement rule-based triage algorithm:
 - Create modules/triage_rules.py
 - Unit tests for edge cases (all severity levels)
3. Create triage API endpoint:

- `POST /api/triage`
 - Input: JSON with symptoms, vitals, demographics
 - Output: KTAS level + description + recommended hospitals
4. Train ML model (optional):
- Clean triage CSV (parse vitals, encode text)
 - Feature engineering + SMOTE for class imbalance
 - Train Random Forest classifier
 - Save model to `models/triage_classifier.pkl`
5. Add ML inference endpoint:
- `POST /api/triage?mode=ml`
 - Load model at startup
 - Clear UI label: "ML Demo Mode"
6. Integrate triage results with hospital search:
- Map KTAS → complaint type
 - Weight score by urgency level
 - Display reasoning in results

Deliverables:

- Functional triage intake form at `/demo.html`
 - Rule-based triage endpoint (production-ready)
 - ML model endpoint (optional, clearly labeled)
 - Enhanced hospital recommendations based on triage
-

Phase 3: Production Readiness (Week 3-4)

Goal: Deploy to Render with optimizations

Tasks:

1. Geocoding optimization:
 - Implement in-memory cache (TTLCache)
 - Add rate limiting (slowapi)
 - Pre-geocode top 1000 US cities → JSON cache
 - Test cache hit rate
2. Environment configuration:
 - Create `.env.example` with all variables
 - Update Dockerfile to accept env vars
 - CORS policy based on `ALLOWED_ORIGINS` env var
3. Structured logging:
 - Replace `print()` with `logging.info/error`
 - JSON-formatted logs for production
 - Add request ID tracking
4. Create `render.yaml` :
 - Web service definition
 - Health check path
 - Environment variables
5. Deployment testing:
 - Deploy to Render (free tier for testing)
 - Verify health checks
 - Test geocoding cache performance

- Load test with `locust` (100 concurrent users)
- 6. Monitoring setup:
 - Uptime monitoring (UptimeRobot or Render dashboard)
 - Error tracking (Sentry free tier)
 - Usage analytics (Plausible or Google Analytics)

Deliverables:

- Deployed app at `hospitracker.onrender.com`
 - Documented deployment guide in README
 - Monitoring dashboards configured
 - Performance benchmarks (p95 latency <500ms)
-

Phase 4: UX Polish (Week 4+)

Goal: Responsive design + accessibility + ranking explanations

Tasks:

1. Responsive design:
 - Mobile-first CSS rewrite
 - Test on iPhone SE, Pixel 5, iPad
 - Collapsible filters on mobile
2. Loading states:
 - CSS skeleton loaders for table
 - Spinner for search button
 - Progress bar for triage form
3. Ranking explanations:
 - Tooltip component (info icons)
 - Modal with detailed methodology
 - "How we rank hospitals" page
4. Accessibility audit:
 - Run axe DevTools scan
 - Fix all critical issues (color contrast, ARIA labels)
 - Keyboard navigation testing
5. Analytics integration:
 - Track page views, searches, triage completions
 - Funnel analysis: Home → Search → View Hospital Details
6. User testing:
 - 5-user usability test sessions
 - Iterate based on feedback

Deliverables:

- Mobile-optimized UI
 - WCAG 2.1 AA compliant
 - Ranking explanation tooltips/modals
 - Analytics dashboard with KPIs
-

Appendix: Quick Reference

Key Files to Modify

File	Purpose	Priority
<code>main.py</code>	Add triage endpoints, update CORS	High
<code>modules/triage_rules.py</code>	Rule-based triage logic	High
<code>modules/triage_ml.py</code>	ML model training + inference	Medium
<code>modules/geolocation.py</code>	Add caching + rate limiting	High
<code>static/home.html</code>	New landing page	High
<code>static/results.html</code>	Enhanced results page	High
<code>static/explore.html</code>	US-wide exploration	Medium
<code>static/demo.html</code>	Triage intake form	High
<code>static/js/api.js</code>	API client wrapper	High
<code>static/js/state.js</code>	URL state management	High
<code>static/css/styles.css</code>	Shared responsive styles	High
<code>render.yaml</code>	Render deployment config	High
<code>Dockerfile</code>	Environment config updates	Medium

New API Endpoints Needed

Endpoint	Method	Purpose	Priority
<code>/api/triage</code>	POST	Rule-based triage	High
<code>/api/triage?mode=ml</code>	POST	ML triage demo	Medium
<code>/api/complaints</code>	GET	List available complaint types	Medium
<code>/api/analytics</code>	POST	Track usage events	Low

Data Files Required

File	Purpose	Size	Status
data/us_er.parquet	Hospital dataset	268KB	✔ Exists
data/triage_ktas.csv	Triage training data	126KB	✔ Uploaded
data/city_geocode_cache.json	Pre-geocoded cities	~50KB	✗ Need to generate
models/triage_classifier.pkl	ML model	~5MB	✗ Need to train
data/complaint_mapping.json	KTAS → Complaint type	<1KB	✗ Need to create

Performance Targets

Metric	Target	Current	Gap
Page load time	<2s	Unknown	Need to measure
API response time (p95)	<500ms	Unknown	Need to measure
Geocoding cache hit rate	>80%	0% (no cache)	Implement caching
Mobile Lighthouse score	>85	Unknown	Need to audit
Uptime	>99%	Unknown	Set up monitoring

Conclusion

HospiTrack has a solid foundation with clean FastAPI architecture, efficient data loading, and working hospital search functionality. The codebase is well-structured and ready for enhancement.

Key Strengths:

- ✔ Production-ready backend (FastAPI + Docker)
- ✔ High-quality hospital dataset (4,088 facilities)
- ✔ Efficient data caching (Parquet format)
- ✔ Smart sorting logic (complaint-adjusted quality)
- ✔ Clean module separation

Critical Next Steps:

1. **Multi-page frontend** (biggest UX gap)
2. **Triage system** (core new feature)
3. **Geocoding optimization** (production blocker)
4. **Render deployment** (user accessibility)

With focused execution on the roadmap above, HospiTrack can evolve from a functional prototype to a production-ready, user-friendly hospital finder with innovative triage capabilities.

Estimated Timeline: 4-6 weeks for full implementation of all priorities.

This analysis provides the foundation for subsequent implementation subtasks. Each recommendation includes specific technical details to guide development.